

Пример 1. Фабричный метод (Factory Method). Новый объект.

```
#include <iostream>
#include <memory>

using namespace std;

class Product;

class Creator
{
public:
    virtual unique_ptr<Product> createProduct() = 0;
};

template <typename Tprod>
class ConCreator : public Creator
{
public:
    virtual unique_ptr<Product> createProduct() override
    {
        return unique_ptr<Product>(new Tprod());
    }
};

#pragma region Product
class Product
{
public:
    virtual ~Product() = 0;
    virtual void run() = 0;
};

Product::~~Product() {}

class ConProd1 : public Product
{
public:
    virtual ~ConProd1() override { cout << "Destructor;" << endl; }
    virtual void run() override { cout << "Method run;" << endl; }
};

#pragma endregion

int main()
{
    shared_ptr<Creator> cr(new ConCreator<ConProd1>());
    shared_ptr<Product> ptr = cr->createProduct();

    ptr->run();
}
```

Пример 2. Фабричный метод (Factory Method). Без повторного создания.

```
#include <iostream>
#include <memory>

using namespace std;

class Product;

class Creator
{
public:
    shared_ptr<Product> getProduct();

protected:
    virtual shared_ptr<Product> createProduct() = 0;

private:
    shared_ptr<Product> product;
};
```

```

template <typename Tprod>
class ConCreator : public Creator
{
protected:
    virtual shared_ptr<Product> createProduct() override
    {
        return shared_ptr<Product>(new Tprod());
    }
};

#pragma region Method Creator
shared_ptr<Product> Creator::getProduct()
{
    if (!product)
    {
        product = createProduct();
    }

    return product;
}

#pragma endregion

#pragma region Product
class Product
{
public:
    virtual ~Product() = 0;
    virtual void run() = 0;
};

Product::~~Product() {}

class ConProd1 : public Product
{
public:
    virtual ~ConProd1() override { cout << "Destructor;" << endl; }
    virtual void run() override { cout << "Method run;" << endl; }
};

#pragma endregion

int main()
{
    shared_ptr<Creator> cr(new ConCreator<ConProd1>());
    shared_ptr<Product> ptr1 = cr->getProduct();
    shared_ptr<Product> ptr2 = cr->getProduct();

    cout << ptr1.use_count() << endl;
    ptr1->run();
}

```

Пример 3. Фабричный метод (Factory Method). Разделение обязанностей.

```

#include <iostream>
#include <memory>
#include <map>

using namespace std;

class Product;

class Creator
{
public:
    virtual unique_ptr<Product> createProduct() = 0;
};

template <typename Tprod>
class ConCreator : public Creator

```

```

{
public:
    virtual unique_ptr<Product> createProduct() override
    {
        return unique_ptr<Product>(new Tprod());
    }
};

#pragma region Product
class Product
{
public:
    virtual ~Product() = 0;
    virtual void run() = 0;
};

Product::~~Product() {}

class ConProd1 : public Product
{
public:
    virtual ~ConProd1() override { cout << "Destructor;" << endl; }
    virtual void run() override { cout << "Method run;" << endl; }
};

#pragma endregion

unique_ptr<Creator> createConCreator()
{
    return unique_ptr<Creator>(new ConCreator<ConProd1>());
}

class Solution
{
public:
    typedef unique_ptr<Creator> (*CreateCreator)();

    bool registration(size_t id, CreateCreator createfun)
    {
        return callbacks.insert(CallBackMap::value_type(id, createfun)).second;
    }
    bool check(size_t id) { return callbacks.erase(id) == 1; }

    unique_ptr<Creator> create(size_t id)
    {
        CallBackMap::const_iterator it = callbacks.find(id);

        if (it == callbacks.end())
        {
            // throw IdError();
        }

        return unique_ptr<Creator>((it->second)());
    }

private:
    using CallBackMap = map<size_t, CreateCreator>;

    CallBackMap callbacks;
};

int main()
{
    Solution solution;

    solution.registration(1, createConCreator);

    shared_ptr<Creator> cr(solution.create(1));
    shared_ptr<Product> ptr = cr->createProduct();
}

```

```
ptr->run();  
}
```

Пример 4. Абстрактная фабрика (Abstract Factory).

```
#include <iostream>  
#include <memory>  
  
using namespace std;  
  
class Image {};  
class Color {};  
  
class BaseGraphics  
{  
public: virtual ~BaseGraphics() = 0;  
};  
BaseGraphics::~BaseGraphics() {}  
  
class BasePen {};  
class BaseBrush {};  
  
class QtGraphics : public BaseGraphics  
{  
public:  
    QtGraphics(shared_ptr<Image> im) { cout << "Constructor QtGraphics;" << endl; }  
    virtual ~QtGraphics() override { cout << "Destructor QtGraphics;" << endl; }  
};  
  
class QtPen : public BasePen {};  
class QtBrush : public BaseBrush {};  
  
class AbstractGraphFactory  
{  
public:  
    virtual unique_ptr<BaseGraphics> createGraphics(shared_ptr<Image> im) = 0;  
    virtual unique_ptr<BasePen> createPen(shared_ptr<Color> cl) = 0;  
    virtual unique_ptr<BaseBrush> createBrush(shared_ptr<Color> cl) = 0;  
};  
  
class QtGraphFactory : public AbstractGraphFactory  
{  
    virtual unique_ptr<BaseGraphics> createGraphics(shared_ptr<Image> im)  
    { return unique_ptr<BaseGraphics>(new QtGraphics(im)); }  
  
    virtual unique_ptr<BasePen> createPen(shared_ptr<Color> cl)  
    { return unique_ptr<BasePen>(new QtPen()); }  
  
    virtual unique_ptr<BaseBrush> createBrush(shared_ptr<Color> cl)  
    { return unique_ptr<BaseBrush>(new QtBrush()); }  
};  
  
int main()  
{  
    shared_ptr<AbstractGraphFactory> grfactory(new QtGraphFactory());  
  
    shared_ptr<BaseGraphics> graphics1 = grfactory->createGraphics(shared_ptr<Image>(new Image()));  
    shared_ptr<BaseGraphics> graphics2 = grfactory->createGraphics(shared_ptr<Image>(new Image()));  
}
```

Пример 5. Строитель (Builder).

```
#include <iostream>  
#include <memory>  
  
using namespace std;  
  
class Product  
{  
public:  
    Product() { cout << "Default constructor;" << endl; }  
    ~Product() { cout << "Destructor;" << endl; }
```

```

        void run() { cout << "Method run;" << endl; }
};

class Builder
{
public:
    virtual bool buildPart1() = 0;
    virtual bool buildPart2() = 0;

    shared_ptr<Product> getProduct();

protected:
    virtual shared_ptr<Product> createProduct() = 0;

    shared_ptr<Product> product;
};

class ConBuilder : public Builder
{
public:
    virtual bool buildPart1() override { cout << "Completed part: " << ++part << ";" << endl; return true; }
    virtual bool buildPart2() override { cout << "Completed part: " << ++part << ";" << endl; return true; }

protected:
    virtual shared_ptr<Product> createProduct() override;

private:
    size_t part{0};
};

class Director
{
public:
    shared_ptr<Product> create(shared_ptr<Builder> builder)
    {
        if (builder->buildPart1() && builder->buildPart2()) return builder->getProduct();

        return shared_ptr<Product>();
    }
};

#pragma region Methods
shared_ptr<Product> Builder::getProduct()
{
    if (!product) { product = createProduct(); }

    return product;
}

shared_ptr<Product> ConBuilder::createProduct()
{
    if (part == 2) { product = shared_ptr<Product>(new Product()); }

    return product;
}

#pragma endregion

int main()
{
    shared_ptr<Builder> builder(new ConBuilder());
    shared_ptr<Director> director(new Director());

    shared_ptr<Product> prod = director->create(builder);

    if (prod)
        prod->run();
}

```

Пример 6. Прототип (Prototype).

```

#include <iostream>
#include <memory>

using namespace std;

class BaseObject
{
public:
    virtual ~BaseObject() = default;

    virtual unique_ptr<BaseObject> clone() = 0;
};

class Object1 : public BaseObject
{
public:
    Object1() { cout << "Default constructor;" << endl; }
    Object1(const Object1& obj) { cout << "Copy constructor;" << endl; }
    ~Object1() { cout << "Destructor;" << endl; }

    virtual unique_ptr<BaseObject> clone() override
    {
        return unique_ptr<BaseObject>(new Object1(*this));
    }
};

int main()
{
    unique_ptr<BaseObject> ptr1(new Object1());

    auto ptr2 = ptr1->clone();
}

```

Пример 7. Одиночка (Singleton).

```

#include <iostream>
#include <memory>

using namespace std;

class Product
{
public:
    static shared_ptr<Product> instance()
    {
        static shared_ptr<Product> myInstance(new Product());

        return myInstance;
    }
    ~Product() { cout << "Destructor;" << endl; }

    void f() { cout << "Method f;" << endl; }

    Product(const Product&) = delete;
    Product& operator=(const Product&) = delete;

private:
    Product() { cout << "Default constructor;" << endl; }
};

int main()
{
    shared_ptr<Product> ptr(Product::instance());

    ptr->f();
}

```

Пример 8. Шаблон одиночка (Singleton).

```

#include <iostream>
#include <memory>

```

```

using namespace std;

template <typename Type>
class Singleton
{
public:
    static Type& instance()
    {
        static unique_ptr<Type> myInstance(new Type());

        return *myInstance;
    }

    Singleton() = delete;
    Singleton(const Singleton<Type>&) = delete;
    Singleton<Type>& operator=(const Singleton<Type>&) = delete;
};

class Product
{
public:
    Product() { cout << "Default constructor;" << endl; }
    ~Product() { cout << "Destructor;" << endl; }

    void f() { cout << "Method f;" << endl; }
};

int main()
{
    Product& d = Singleton<Product>::instance();

    d.f();
}

```

Пример 9. Пул объектов (Object Pool).

```

#include <iostream>
#include <memory>
#include <iterator>
#include <vector>

using namespace std;

class Product
{
private:
    static size_t count;
public:
    Product() { cout << "Constructor(" << ++count << ");" << endl; }
    ~Product() { cout << "Destructor(" << count-- << ");" << endl; }

    void clear() { cout << "Method clear: 0x" << this << endl; }
};

size_t Product::count = 0;

template <typename Type>
class ObjectPool
{
public:
    static shared_ptr<ObjectPool<Type>>> instance();

    shared_ptr<Type> getObject();
    bool releaseObject(shared_ptr<Type>& obj);
    size_t count() const { return pool.size(); }

    iterator<output_iterator_tag, const pair<bool, shared_ptr<Type>>>> begin() const;
    iterator<output_iterator_tag, const pair<bool, shared_ptr<Type>>>> end() const;

    ObjectPool(const ObjectPool<Type>&) = delete;
    ObjectPool<Type>& operator=(const ObjectPool<Type>&) = delete;
}

```

```

private:
    vector<pair<bool, shared_ptr<Type>>> pool;

    ObjectPool() {}

    pair<bool, shared_ptr<Type>> create();

    template <typename Type>
    friend ostream& operator << (ostream& os, const ObjectPool<Type>& pl);
};

#pragma region ObjectPool class Methods
template <typename Type>
shared_ptr<ObjectPool<Type>> ObjectPool<Type>::instance()
{
    static shared_ptr<ObjectPool<Type>> myInstance(new ObjectPool<Type>());

    return myInstance;
}

template <typename Type>
shared_ptr<Type> ObjectPool<Type>::getObject()
{
    size_t i;
    for (i = 0; i < pool.size() && pool[i].first; ++i);

    if (i < pool.size())
    {
        pool[i].first = true;
    }
    else
    {
        pool.push_back(create());
    }

    return pool[i].second;
}

template <typename Type>
bool ObjectPool<Type>::releaseObject(shared_ptr<Type>& obj)
{
    size_t i;
    for (i = 0; i < pool.size() && pool[i].second != obj; ++i);

    if (i == pool.size()) return false;

    obj.reset();
    pool[i].first = false;
    pool[i].second->clear();

    return true;
}

template <typename Type>
pair<bool, shared_ptr<Type>> ObjectPool<Type>::create()
{
    return pair<bool, shared_ptr<Type>>(true, shared_ptr<Type>(new Type()));
}

#pragma endregion

template <typename Type>
ostream& operator << (ostream& os, const ObjectPool<Type>& pl)
{
    for (auto elem : pl.pool)
        os << "{" << elem.first << ", 0x" << elem.second << "} ";

    return os;
}

int main()

```



```
{  
    shared_ptr<ObjectPool<Product>> pool = ObjectPool<Product>::instance();  
  
    vector<shared_ptr<Product>> vec(4);  
  
    for (auto& elem : vec)  
        elem = pool->getObject();  
  
    pool->releaseObject(vec[1]);  
  
    cout << *pool << endl;  
  
    shared_ptr<Product> ptr = pool->getObject();  
    vec[1] = pool->getObject();  
  
    cout << *pool << endl;  
}
```